

Prática — MongoDB com Pokémon

15 exercícios de CRUD e agregação (mongosh)

Unidade 7 · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

O dataset

- Base pública pogoapi.net — cada documento é um Pokémon com `pokemon_name`, `form`, `pokedex_height`, `pokedex_weight`, `pokemon_id`, `buddy_scale` ...
- Rode na **VM LabDatabase** (MongoDB via Docker), com **mongosh**.

```
// 0 – criar a collection a partir do JSON
// baixe: https://pogoapi.net/api/v1/pokemon_height_weight_scale.json
db.pokemon.insertMany( /* array de documentos do JSON */ )
```



Tente cada exercício antes de olhar a solução no slide seguinte.

Busca e contagem (1–3)

```
// 1 – o Pokémon "Ninetales"  
db.pokemon.find({ pokemon_name: "Ninetales" })  
  
// 2 – quantos documentos com "Pikachu"  
db.pokemon.find({ pokemon_name: "Pikachu" }).count()  
  
// 3 – Pikachu cuja "form" termina em "star" (regex)  
db.pokemon.find({ pokemon_name: "Pikachu", form: /star$/ })
```

💡 `/star$/` é uma **expressão regular** — o `$` ancora no fim.

Filtros numéricos (4–6)

```
// 4 – altura = 1 e peso < 0.5
db.pokemon.find({ pokedex_height: 1.0, pokedex_weight: { $lt: 0.5 } })

// 5 – altura < 1.0
db.pokemon.find({ pokedex_height: { $lt: 1.0 } })

// 6 – contar o resultado anterior
db.pokemon.find({ pokedex_height: { $lt: 1.0 } }).count()
```

💡 Operadores de comparação: `$lt`, `$lte`, `$gt`, `$gte`, `$ne`.

Atualizar, remover, recriar (7–9)

```
// 7 – Bulbasaur: alterar a altura para 10.0
db.pokemon.updateMany(
  { pokemon_name: "Bulbasaur" },
  { $set: { pokedex_height: 10.0 } }
)

// 8 – remover todos os "Pikachu"
db.pokemon.deleteMany({ pokemon_name: "Pikachu" })

// 9 – apagar e recriar a collection
db.pokemon.drop()
db.pokemon.insertMany( /* dataset novamente */ )
```

`updateMany` / `deleteMany` afetam **todos** os documentos que casam — confira o filtro antes.

Projeção — escolher campos (11–13)

```
// 11 — altura entre 0.3 e 0.75; só o nome, sem _id
db.pokemon.find(
  { pokedex_height: { $lte: 0.75, $gte: 0.3 } },
  { pokemon_name: 1, _id: 0 })

// 12 — form "Normal"; alguns campos
db.pokemon.find({ form: "Normal" },
  { buddy_scale: 1, form: 1, pokemon_id: 1, pokemon_name: 1, _id: 0 })

// 13 — form começando com "Fall"; id, pokemon_id e nome
db.pokemon.find({ form: /^Fall/ }, { pokemon_id: 1, pokemon_name: 1 })
```



Projeção: 1 inclui, 0 exclui. `_id` vem por padrão — use `_id: 0` para omitir.

Agregação (10 e 14)

```
// 10 – somar as alturas de todos os Pokémon
db.pokemon.aggregate([
  { $group: { _id: null, TotalAltura: { $sum: "$pokedex_height" } } }
])

// 14 – agrupar por nome: somar pesos e contar repetidos
db.pokemon.aggregate([
  { $group: {
    _id: "$pokemon_name",
    Total: { $sum: "$pokedex_weight" },
    Quant: { $sum: 1 }
  } }
])
```

💡 \$group é o GROUP BY do MongoDB; _id define a chave do grupo (null = tudo num grupo só).

O que você praticou

- **CRUD:** `find`, `insertMany`, `updateMany`, `deleteMany`, `drop`.
- **Filtros:** igualdade, comparação (`$lt` / `$gte`), **regex** (`/star$/` , `/^Fall/`).
- **Projeção:** incluir/excluir campos, omitir `_id`.
- **Agregação:** `$group` com `$sum` e contagem.

 Compare com o SQL equivalente no deck "**De SQL para MongoDB**".

Fim da prática

Dúvidas? howard.cruz@faesa.br

 **Voltar ao índice**
todas as unidades